Algorithm Exam Cheatsheet

LECTURES 0–6 · BFS · DFS · CYCLES · EXHAUSTIVE SEARCH · BACKTRACKING

STEP 0 — CHOOSE YOUR DATA TYPES

Data type decision table

VARIABLE	VERTICES 0…N-1	ARBITRARY LABELS	PURPOSE
`visited`	`[False]*n`	`set()`	Have I seen this vertex?
`queue`	`[]` → `.append()` + `.pop(0)`		BFS frontier (FIFO)
`stack`	`[]` → `.append()` + `.pop()`		DFS frontier (LIFO)
`distance`	`[-1]*n`	`{start: 0}`	Shortest edge count from start
`parent`	`[None]*n`	`{start: None}`	Reconstruct path
`in_stack`	`set()`		Current recursion path (directed cycles)
`partial`	`list` (ordered) · `dict` (assignment) · `set` (membership)		Backtracking state

> **Adj list** → `for w in graph[v]` — Θ(V+E) | **Adj matrix** → `for w in range(n): if graph[v][w]==1` — Θ(V²)

GRAPH TRAVERSAL TEMPLATES

BFS `QUEUE · FIFO`

USE WHEN

Reachability · connectedness · shortest path (unweighted) · layer-by-layer

DATA

`visited` `queue` `+ distance?` `+ parent?`

```python
def bfs(graph, start):
    visited = set([start])
    queue = [start]

    while queue:
        cur = queue.pop(0)
        for w in graph[cur]:
            if w not in visited:
                visited.add(w)
                queue.append(w)
    return visited
```

BFS + Distance

USE WHEN

Shortest path count · distance from source · "how many edges"

```python
def bfs_dist(graph, start):
    visited = set([start])
    dist = {start: 0}
    queue = [start]

    while queue:
        cur = queue.pop(0)
        for w in graph[cur]:
            if w not in visited:
                visited.add(w)
                dist[w] = dist[cur] + 1
                queue.append(w)
    return dist
```

DFS Recursive `IMPLICIT STACK`

USE WHEN

Traversal · components · cycle detection · path exploration

```python
def dfs_all(graph):
    visited = set()

    def dfs(v):
        visited.add(v)
        for w in graph[v]:
            if w not in visited:
                dfs(w)

    for v in graph:        # handles disconnected
        if v not in visited:
            dfs(v)
    return visited
```

DFS Iterative `EXPLICIT STACK · LIFO`

USE WHEN

Asked for iterative / non-recursive DFS · explicit control

```python
def dfs_iter(graph, start):
    visited = set()
    stack = [start]

    while stack:
        cur = stack.pop()
        if cur not in visited:
            visited.add(cur)
            for w in graph[cur]:
                if w not in visited:
                    stack.append(w)
    return visited
```

CYCLE DETECTION

Directed Cycle VISITED + IN_STACK

KEY INSIGHT

`visited` = seen ever · `in_stack` = on current path → edge to in_stack = cycle

```python
def has_dir_cycle(graph):
    visited = set()
    in_stack = set()

    def dfs(v):
        visited.add(v)
        in_stack.add(v)
        for w in graph[v]:
            if w not in visited:
                if dfs(w): return True
            elif w in in_stack:
                return True
        in_stack.remove(v)
        return False

    for v in graph:
        if v not in visited:
            if dfs(v): return True
    return False
```

Undirected Cycle VISITED + PARENT

KEY INSIGHT

Pass parent so we skip the edge we came from. Neighbour visited & ≠ parent → cycle

```python
def has_undir_cycle(graph):
    visited = set()

    def dfs(v, parent):
        visited.add(v)
        for w in graph[v]:
            if w not in visited:
                if dfs(w, v): return True
            elif w != parent:
                return True
        return False

    for v in graph:
        if v not in visited:
            if dfs(v, None): return True
    return False
```

EXHAUSTIVE SEARCH & BACKTRACKING

Exhaustive Search TRY EVERYTHING

IDENTIFY FIRST

State → where am I? · **Action** → how to move? · **Goal** → when complete? · **Evaluate** → what to do with result?

3 RETURN STYLES

Existence: return on first valid · **Counting**: sum valid · **Optimization**: track best

SUBSETS

```python
def search(i, chosen):
    if i == n:
        evaluate(chosen); return
    search(i+1, chosen)          # skip
    chosen.append(items[i])
    search(i+1, chosen)          # take
    chosen.pop()
```

PERMUTATIONS

```python
def search(cur, used):
    if len(cur) == n:
        evaluate(cur); return
    for i in range(n):
        if i not in used:
            used.add(i)
            cur.append(items[i])
            search(cur, used)
            cur.pop()
            used.remove(i)
```

Backtracking PRUNE EARLY

VS EXHAUSTIVE

Same structure, but **skip branches whose partial state is already impossible**

IDENTIFY

1. Partial solution · 2. Choices · 3. Constraint check · 4. Undo step

LIST-BASED (PATH / QUEENS)

```python
def solve(data):
    partial = []
    def bt():
        if complete(partial):
            return partial.copy()
        for c in choices(partial):
            if valid(c, partial):
                partial.append(c)
                ans = bt()
                if ans is not None:
                    return ans
                partial.pop()
        return None
    return bt()
```

DICT-BASED (COLORING / SAT)

```python
def solve(verts, colors):
    asgn = {}
    def bt(idx):
        if idx == len(verts):
            return asgn.copy()
        v = verts[idx]
        for c in colors:
            if valid(v, c):
                asgn[v] = c
                ans = bt(idx+1)
                if ans is not None:
                    return ans
                del asgn[v]
        return None
    return bt(0)
```

Reading the question → choosing the template

GRAPH QUESTION CHECKLIST

1. Adj list or adj matrix?
2. BFS or DFS?
3. Recursive or iterative?
4. Need distance / parent / in_stack?
5. Connected or disconnected? (→ outer loop)
6. Existence / traversal / cycle?

BACKTRACKING CHECKLIST

1. What is the partial solution?
2. What are the choices?
3. What is the constraint?
4. How to undo?
5. Return one / all / True-False?

QUICK PATTERN MATCH

WORDING	→ TEMPLATE
shortest path / min edges	BFS + distance
reachable / connected	BFS or DFS
reconstruct path	BFS + parent
directed cycle / dependency	DFS + in_stack
undirected cycle	DFS + parent arg
all subsets / permutations	Exhaustive search
valid assignment + constraints	Backtracking
coloring / n-queens / SAT	Backtracking
Hamiltonian / TSP	Backtracking or exhaustive

> **Queue** = FIFO = BFS = layer-by-layer
> **Stack** = LIFO = DFS = go deep then backtrack
> **Recursive DFS** = Python call stack does the LIFO for you